TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO

Báo cáo Chuyên đề

Môn: Lập trình Đa nền tảng

«Chủ đề 03: Đa luồng với Isolates»

Tối ưu hiệu năng xử lý tác vụ nặng không nghẽn luồng giao diện

SVTH: 1. Lê Anh Khoa
2. Ngô Quang Thế

Đà Nẵng, 2026

Phân công công việc

TT	HỌ VÀ TÊN	VAI TRÒ & NỘI DUNG CÔNG VIỆC THEO WBS	ĐÓNG GÓP
1	**Lê Anh Khoa**	• **Business Logic (Chính):** Thuật toán bóc tách JSON lớn & tính toán FLOPS (`SensorRecord`); Hiện thực đa luồng với `compute()` và `Isolate.spawn()` + Ports; Kỹ thuật Throttling và chống rò rỉ RAM. • **Database / File I/O:** Xây dựng `JsonGeneratorService` sinh & đọc ngầm tập tin 16.5MB – 132.1MB; Tối ưu luồng ghi đĩa với `flush()`. • **Báo cáo & Git:** Quản trị Git Repo; Soạn thảo Báo cáo phần *Cơ sở lý thuyết Event Loop / Isolate & Kiến trúc hệ thống*.	50%
2	**Ngô Quang Thế**	• **UI Design (Chính):** Thiết kế `BenchmarkScreen` (Bảng điều khiển, slider tải) và `FpsMonitorPanel` (Hoạt ảnh bánh răng, biểu đồ FPS thời gian thực). • **State Management & Tích hợp:** Quản lý state luồng dữ liệu tiến độ (0% – 100%) và thuật toán bắt treo UI (`freezeMs`); **Tích hợp gọi Isolate vào UI**. • **Slide & Live Demo:** Thiết kế Slide báo cáo; Soạn thảo Báo cáo phần *Thiết kế Giao diện & Kịch bản thực nghiệm*; Trực tiếp thực hiện Live Demo trên Windows.	50%

Nội dung

I. Bản chất Công nghệ Cốt lõi

- Bài toán thực tế: Hiện tượng đơ giao diện (UI Freeze).
- Mô hình Single-thread & Event Loop trong Dart.
- Multi-thread Isolate & Bộ nhớ độc lập (Isolated Heap).
- Giao tiếp qua Port & So sánh `compute()` vs `Isolate.spawn()`.

II. Kiến trúc Giải pháp & Luồng Dữ liệu

- Thiết kế bài toán thực nghiệm trên Windows Desktop.
- Sơ đồ kiến trúc 3 tầng & Sơ đồ luồng dữ liệu 1, 2.

III. Hiện thực Mã nguồn & Kết quả Thực nghiệm

- Cấu trúc thư mục chuẩn Clean Architecture.
- Thuật toán đo FPS thời gian thực bằng Ticker.
- Xử lý ngoại lệ & Chống rò rỉ RAM.
- Bảng thông số đo đạc thực tế ở mức 132.1 MB.

IV. Live Demo, Tổng kết & Tài liệu tham khảo

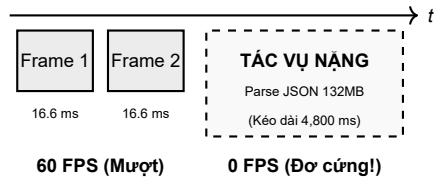
Bài toán thực tế — Vì sao giao diện Flutter bị đơ (UI Freeze)?

1. Quy chuẩn Khung hình Mượt mà (60Hz / 120Hz):

- Ở màn hình 60Hz: Quỹ thời gian $\Delta t = \frac{1000 \text{ ms}}{60} \approx \mathbf{16.6}$ ms để hoàn tất: *Build* → *Layout* → *Paint*.
- Ở màn hình 120Hz (ProMotion): Quỹ thời gian chỉ còn $\mathbf{8.3}$ ms.

2. Hiện tượng Jank & UI Freeze:

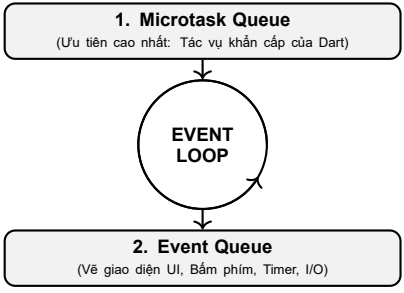
- **Jank (Giật hình):** Thời gian xử lý vượt quá 16.6ms → Rớt khung hình.
- **UI Freeze (Đơ cứng):** Tác vụ CPU-bound nặng chiếm dụng Main Thread liên tục nhiều giây → Giao diện rơi vào trạng thái mất phản hồi (ANR), chỉ số FPS sụt giảm về 0.



Mô hình Single-thread & Bản chất Event Loop trong Dart

1. Cấu trúc Vòng lặp Sự kiện (Event Loop):

- **Microtask Queue:** Xử lý tác vụ nội bộ khẩn cấp của Dart trước.
- **Event Queue:** Xử lý sự kiện chạm, vẽ màn hình, Timer, I/O.



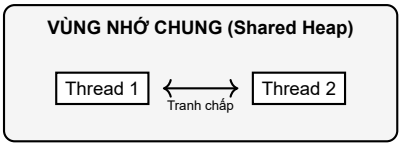
2. Điểm mấu chốt: Vì sao `async / await` trên Main Thread vẫn đơ?

- `async / await` hoạt động theo cơ chế đa nhiệm cộng tác (Cooperative Multitasking) **trên CÙNG 1 LUỒNG**.
- Nó chỉ nhường quyền thực thi luồng khi chờ đợi các tác vụ I/O (Non-Blocking I/O).
- Với tác vụ **CPU-bound** (`jsonDecode`, vòng lặp toán học), CPU liên tục làm việc 100% → Event Loop bị khóa cứng!

Multi-thread với Isolate & Bộ nhớ độc lập (Isolated Heap)

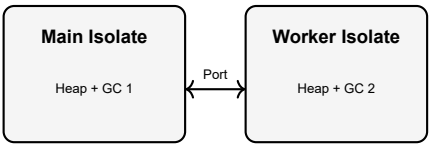
Thread truyền thống (Java, C++, C#):

- Các luồng **chia sẻ chung một vùng nhớ Heap**.
- Dễ xảy ra tranh chấp dữ liệu (**Race Condition**).
- Bắt buộc dùng cơ chế khóa (**Lock / Mutex**) → Nguy cơ cao bị khóa chết (**Deadlock**).



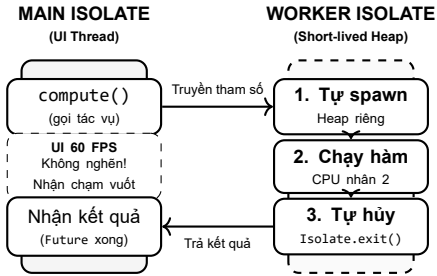
Isolate trong Dart (Actor Model):

- Mỗi Isolate có **vùng nhớ Heap riêng biệt**.
- Mỗi Isolate có bộ thu gom rác (**GC**) riêng.
- **Triệt tiêu 100% Race Condition**: Không cần Lock/Mutex.
- Giao tiếp qua **Truyền thông điệp (Message)**.



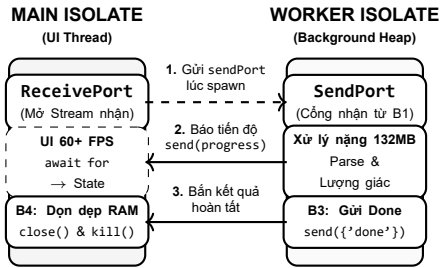
Cơ chế hoạt động của hàm `compute()`

1. **Bản chất: Short-lived Isolate (Dùng một lần)**
   - Hàm tiện ích cao cấp của Flutter (`foundation.dart`).
   - Mô hình **One-shot (Fire-and-forget)**: Tự động khởi sinh để giải quyết 1 tác vụ rồi tự hủy.
2. **Chu trình 4 bước bên dưới (Under the hood):**
   - **B1 (Spawn ngầm):** Khởi tạo Isolate phụ & Heap độc lập với một cổng phản hồi đơn (`SingleResponsePort`).
   - **B2 (Chuyển giao):** Truyền con trỏ hàm Top-level và dữ liệu đầu vào sang Isolate mới.
   - **B3 (Thực thi):** Chạy trên nhân CPU riêng biệt → Main Thread hoàn toàn rảnh tay duy trì **60+ FPS**.
   - **B4 (Exit & Tự hủy):** Gọi `Isolate.exit()` trả kết quả về Main và **thu hồi 100% RAM ngay lập tức**.
3. **Quy tắc bắt buộc:**
   - Hàm truyền vào phải là **Top-level** hoặc **static method** (không được bắt giữ closure hay con trỏ `this`).



Cơ chế giao tiếp giữa các Isolate qua SendPort & ReceivePort

1. **Bản chất cặp Port (Mô hình Actor trong Dart):**
   - **`ReceivePort`:** Hộp thư nhận tin gắn với Isolate tạo ra nó. Hoạt động như một `Stream<dynamic>` bất đồng bộ, chỉ đọc.
   - **`SendPort`:** Cổng gửi tin có thể truyền xuyên biên giới Isolate (Serializable). Cho phép mô hình Nhiều gửi – Một nhận (Many-to-One).
2. **Chu trình giao tiếp 4 bước (Handshake Protocol):**
   - **B1 (Bắt tay):** Main tạo `ReceivePort`, gửi `sendPort` làm tham số khởi tạo qua lệnh `Isolate.spawn()`.
   - **B2 (Báo tiến độ):** Worker tính toán nặng, định kỳ gửi bản tin `sendPort.send({'progress': x})`.
   - **B3 (Cập nhật UI):** Main dùng `await for` lắng nghe Stream → gọi `setState()` giữ vững **60+ FPS**.
   - **B4 (Dọn dẹp):** Nhận tin `done` → gọi `rp.close()` và `iso.kill()` để **triệt tiêu nguy cơ rò rỉ RAM**.
3. **Nguyên lý cốt lõi:** Không chia sẻ bộ nhớ (No Shared Memory) → **Triệt tiêu 100% Race Condition / Deadlock**.



So sánh — Hàm `compute()` vs `Isolate.spawn()`

Tiêu chí so sánh	Hàm `compute()` (Foundation)	`Isolate.spawn()` + Ports (Core)
Vòng đời (Lifecycle)	**Ngắn hạn (Short-lived):** Tự sinh ra, chạy 1 hàm duy nhất rồi tự hủy.	**Dài hạn (Long-lived):** Lập trình viên chủ động quản lý vòng đời và thời điểm hủy.
Khả năng báo tiến độ	**Chỉ 1 lần:** Trả kết quả khi xong hoàn toàn (Fire-and-forget).	**Thời gian thực:** Truyền phát liên tục (Stream) các gói tin tiến độ (0% → 100%) qua SendPort.
Độ phức tạp mã nguồn	**Cực kỳ đơn giản:** Chỉ 1 dòng lệnh: `await compute(task, args);`	**Trung bình:** Phải mở ReceivePort, lắng nghe Stream và gọi lệnh `kill()`.
Tình huống tối ưu	Tác vụ nhanh gọn 1 lần: Parse JSON API, nén ảnh, giải mã dữ liệu.	Tác vụ kéo dài nhiều giây cần thanh tiến trình ProgressBar trực quan.

Thiết kế Bài toán Thực nghiệm trên Windows Desktop

1. Thách thức với Vi xử lý mạnh (Intel Core i7-13th H):

- Xung nhịp đơn nhân Turbo đạt tới $\mathbf{5.0}$ GHz.
- Nếu chỉ parse JSON 15MB thông thường: chỉ mất ~ 180 ms, giao diện chỉ giật nhẹ, không kịp thấy đơ!

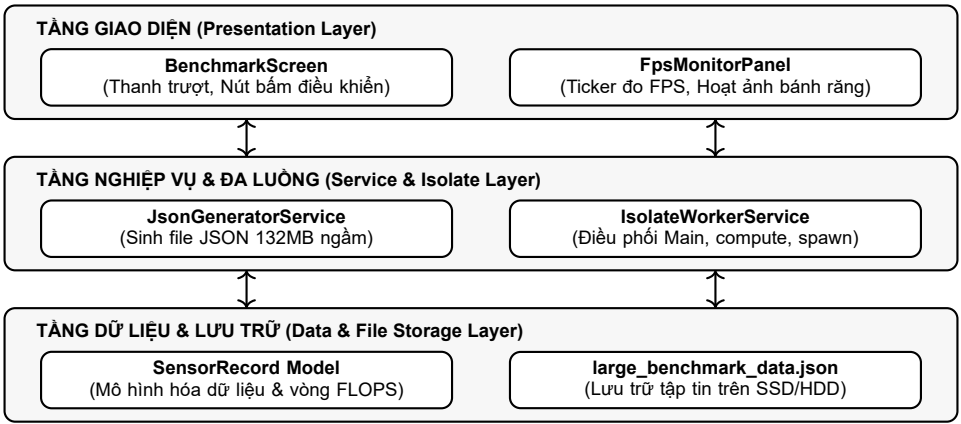
2. Giải pháp Nhân tải Đa tầng:

- **Dung lượng file lớn:** Tùy biến từ **16.5 MB** (60.000 bản ghi) đến **132.1 MB** (480.000 bản ghi cảm biến).
- **Tải tính toán FLOPS:** Lặp 25 lần tính toán lượng giác ($\sqrt{x}, \sin, \cos$) trên từng bản ghi.
- Tạo tải tính toán đủ lớn để làm nghẽn (freeze) Main Thread $\mathbf{3 - 8}$ giây.

3. Vì sao chọn Windows Desktop thay vì Web?

- **Hạn chế Sandbox trên Web:** Môi trường trình duyệt cô lập bảo mật, không cho phép truy cập trực tiếp hệ thống tệp cục bộ (`dart:io`).
- **Web không có Isolate thật sự:** Trình duyệt biên dịch sang JavaScript đơn luồng, không hỗ trợ `dart:isolate`. Hàm `compute()` trên Web tự fallback về Main Thread.
- **Windows Native Desktop:** Máy ảo Dart VM thực thi đa nhân x86_64 thực thụ.

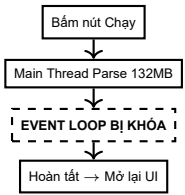
Sơ đồ Kiến trúc Hệ thống (System Architecture Diagram)



Sơ đồ Luồng dữ liệu (Data Flow 1) — Main Thread vs Hàm `compute()`

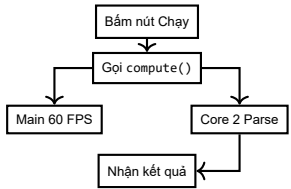
Luồng 1: Chạy trực tiếp trên Main Thread

- Bấm nút → Gọi trực tiếp trên luồng chính.
- **Event Loop bị khóa cứng** suốt 4,800 ms.
- **Hậu quả:** Toàn bộ tương tác UI bị đóng băng, thao tác trên Slider bị vô hiệu hoá.

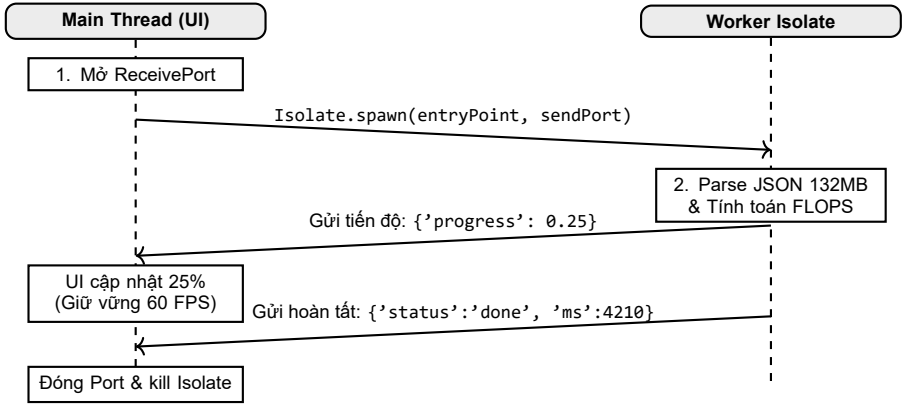


Luồng 2: Tách luồng bằng `compute()`

- Bấm nút → Gọi `compute(heavyParseTask, ...)`.
- Task chuyển sang **nhân CPU số 2** xử lý độc lập.
- **Kết quả:** Main Thread rảnh tay, **giữ vững 60+ FPS**!



Sơ đồ Luồng dữ liệu (Data Flow 2) — `Isolate.spawn` + Ports



- **Kỹ thuật Throttling:** Chia tổng số bước lặp $\div 20$, chỉ gửi đúng 20 bản tin để chống nghẽn hàng đợi tin nhắn (*SendPort Flooding*).

Cấu trúc thư mục chuẩn & Thuật toán đo FPS (Ticker)

Cấu trúc thư mục Clean Architecture:

- **lib/main.dart**:
  Khởi tạo & khởi chạy ứng dụng.
- **lib/models/sensor_record.dart**:
  Định nghĩa bản ghi và vòng lặp FLOPS.
- **lib/services/json_generator_service.dart**:
  Sinh file JSON ngầm bằng StringBuffer.
- **lib/services/isolate_worker_service.dart**:
  Điều phối 3 chế độ chạy Isolate.
- **lib/widgets/fps_monitor_panel.dart**:
  Bộ đo FPS thời gian thực và biểu đồ cột.
- **lib/screens/benchmark_screen.dart**:
  Màn hình điều khiển và bảng kết quả.

Thuật toán đo FPS & Bắt nghẽn UI:

- **Ticker gắn Vsync:** Lắng nghe từng chu kỳ quét màn hình từ phần cứng.
- **Đo FPS mỗi 250ms:**

$$\text{FPS} = \left(\frac{\text{frameCount} \times 1000}{\Delta t_{\text{ms}}} \right).\text{clamp}(0, 240)$$

- **Thuật toán phát hiện nghẽn UI:**

```
if (gapMs > 250) {
  _freezeMs = gapMs; _currentFps = 0;
  // Ghi nhận số ms đơ & cập nhật UI
}
```

Xử lý Ngoại lệ Đa Luồng & Quản lý Tài Nguyên Bộ nhớ

1. Bắt lỗi an toàn qua ranh giới Port:

- Bọc toàn bộ logic Worker Isolate trong `try / catch`.
- Nếu xảy ra lỗi: Gửi gói tin `{'type':'error'}` về Main Thread.
- Main Thread đóng Port an toàn, ngăn chặn hiện tượng rò rỉ tài nguyên và treo luồng tiếp nhận (Hang UI).

2. Bắt buộc `await flush()` khi ghi file:

- Bắt buộc có `await sink.flush()` và `await sink.close()` trước khi thoát Isolate để đảm bảo đẩy toàn bộ dữ liệu từ bộ đệm (buffer) ghi hoàn tất xuống ổ đĩa trước khi huỷ tiến trình.

3. Giải phóng tài nguyên chống rò rỉ RAM:

- Khi hoàn tất hoặc gặp lỗi, lập tức gọi:
  - `receivePort.close();` → Đóng Stream.
  - `workerIsolate.kill();` → Tiêu hủy Isolate tức thì, giải phóng Heap về hệ điều hành.

4. Ngăn chặn Compiler tối ưu hóa:

- Sử dụng biến tích luỹ kiểm tra điều kiện (`if (acc == 123456789.987)`) nhằm đảm bảo trình biên dịch không được bỏ qua vòng lặp tính toán (*Dead-code elimination*).

Kết quả Thực nghiệm & Bảng đối chiếu Thông số

Chỉ số Đánh giá	Main Thread	Hàm `compute()`	`Isolate.spawn()` + Ports
Chỉ số FPS trung bình	**0 FPS** (Đơ cứng)	**60+ FPS** (Mượt mà)	**60+ FPS** (Mượt mà)
Thời gian đơ UI (Freeze)	**13,174 ms**	**0 ms**	**0 ms**
Hoạt ảnh Bánh răng	Đứng hình hoàn toàn	Xoay đều liên tục	Xoay đều liên tục
Thanh trượt Test Slider	Bị khóa cứng	Kéo thả mượt mà	Kéo thả mượt mà
Cập nhật Progress Bar	Không thể cập nhật	Nhảy thẳng 100%	Tịnh tiến 0% $\rightarrow$ 100%
Thời gian thực thi	13,174 ms	13,653 ms	13,590 ms

- **Đánh giá:** Dữ liệu ở mức tải 8x (480.000 bản ghi – 132.1 MB) chứng minh rõ rệt: Hai giải pháp Isolate giúp giải phóng hoàn toàn Main Thread, duy trì giao diện mượt mà 60 FPS.

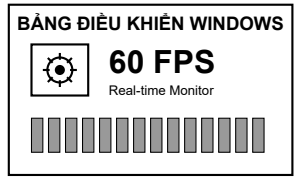
Minh chứng Thực nghiệm: Giao diện Ứng dụng Thực tế

Giao diện ứng dụng Isolate FPS Benchmark & Bộ đo khung hình thời gian thực trên Windows:



Chuyển sang Live Demo thực tế trên ứng dụng Windows

1. **Khởi tạo dữ liệu:** Sinh file JSON 132.1 MB ngầm trong Isolate (không đơ UI lúc chuẩn bị dữ liệu).
2. **Kiểm chứng Main Thread:**
   - Bấm chạy → Bánh răng lập tức đứng khựng.
   - Kéo Test Slider → Bị khóa đơ hoàn toàn.
   - Bộ đo phát hiện và hiện cột đỏ **0 FPS** trong một khoảng thời gian!
3. **Kiểm chứng `compute()`:** Bấm chạy → Bánh răng vẫn xoay mượt mà ở **60+ FPS**.
4. **Kiểm chứng `Isolate.spawn()`:** Bấm chạy → Bánh răng vẫn xoay mượt mà ở **60+ FPS** và thanh tiến trình tăng dần 0% → 100% theo thời gian thực.



(Nhóm xin phép chuyển sang cửa sổ ứng dụng Windows để thực hiện Live Demo trực tiếp)

Giải thích các đoạn Mã nguồn Then chốt

1. Chạy trên Main Thread (Đơ UI):

```
ParseBenchmarkResult runOnMainThread(...) {
  return heavyParseTask({
    'filePath': path,
    'intensity': intensity,
  });
}
```

2. Chạy bằng compute() (Gọn nhẹ):

```
Future<ParseBenchmarkResult> runWithCompute
     (...) async {
  return await compute(heavyParseTask, {
    'filePath': path,
    'intensity': intensity,
  });
}
```

3. Chạy bằng Isolate.spawn (Báo tiến độ):

```
ReceivePort receivePort = ReceivePort();
Isolate iso = await Isolate.spawn(
  spawnIsolateEntry,
  {'sendPort': receivePort.sendPort, ...}
);

await for (var msg in receivePort) {
  if (msg['type'] == 'progress') {
    onProgress(msg['progress']);
  } else if (msg['type'] == 'done') {
    receivePort.close();
    iso.kill(priority: Isolate.immediate);
    return res;
  }
}
```

Tổng kết & 3 Quy tắc vàng khi sử dụng Isolate

3 QUY TẮC VÀNG KHI LỰA CHỌN GIẢI PHÁP ĐA LUỒNG TRONG FLUTTER:

1. **Quy tắc 1 (I/O thông thường):**
   Nếu chỉ là gọi HTTP API, đọc file nhỏ vài KB hoặc truy vấn SQLite → **Dùng `async / await`**, KHÔNG dùng Isolate vì chi phí khởi tạo luồng lớn hơn thời gian chạy!
2. **Quy tắc 2 (Tác vụ nặng chạy 1 lần):**
   Nếu cần bóc tách JSON lớn từ mạng, nén ảnh, xử lý dữ liệu một lần → **Dùng hàm `compute()`** (gọn nhẹ 1 dòng lệnh, tự sinh tự hủy, an toàn bộ nhớ).
3. **Quy tắc 3 (Tác vụ nặng kéo dài cần báo tiến độ):**
   Nếu tác vụ kéo dài nhiều giây cần cập nhật thanh tiến trình ProgressBar liên tục → **Dùng `Isolate.spawn()` kết hợp SendPort / ReceivePort**.

Tài liệu tham khảo

Tài liệu kỹ thuật chính thức & Tài liệu tham khảo:

1. **Google Flutter Documentation (2026):** *Concurrency and isolates in Dart*, Official Guides. `https://dart.dev/guides/language/concurrency`
2. **Dart VM Core Library:** Thư viện chuẩn `dart:isolate`, `dart:async`, và mã nguồn hàm `compute()` trong `package:flutter/foundation.dart`.
3. **Flutter Performance Guide:** *Profiling Flutter apps and diagnosing dropped frames (Build, Layout, Paint)*, Flutter Architecture Overview.
4. **Kho lưu trữ mã nguồn Đồ án (GitHub):** Dự án mã nguồn mở `isolate_fps_benchmark` trên nền tảng Windows Desktop Native. `https://github.com/KhoaLe246205/cross-platform/tree/main/isolate`

XIN CHÂN THÀNH CẢM ƠN!

QUÝ THẦY VÀ CÁC BẠN ĐÃ LẮNG NGHE